Spark Project

January 14, 2026

1 Names of the members of the group

Daria-Teodora GODINEL and Dana-Georgiana TOMA

2 Introduction

In this work, we use publicly accessible datasets that are available from major cloud providers for studying large-scale cluster activity. The Google Cluster Data, which offers detailed information about machine events, task execution, resource requests and actual resource usage over long periods of time, is our main focus.

We used Apache Spark to develop our analysis pipeline. All tests were performed on our subset of the data. Using a Dataproc cluster, we expanded the study by implementing the identical pipeline on Google Cloud Platform.

We looked at other traces from Microsoft Azure in addition to the Google datasets to compare serverless execution models with infrastructure-oriented cluster management. By using a comparative approach, we could find basic differences in how contemporary cloud platforms manage task execution and resource allocation across different architectures and time periods.

3 Data Analysis Methodology

3.1 Questions

In this section, we will present our results of the analysis. We run our code on our sample (part 105 - 109).

Q1. What is the distribution of the machines according to their CPU capacity? Can you explain (motivate) it?

```
Q1: Machine distribution by CPU capacity
+----+-----+
|CPUs|count|
+----+-----+
|NULL|   32|
|0.25|  126|
| 0.5|11659|
| 1.0|  798|
+----+-----+
```

Figure 1: Result Q1

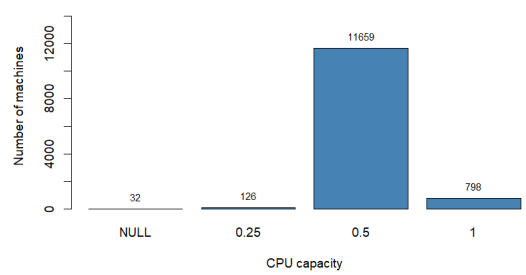

Figure 2: Distribution of machines by CPU capacity

Figure 2 shows the distribution of machines according to their CPU capacity. The analysis of machine CPU capacities shows an uneven distribution. The majority of machines, 11,659, have a CPU capacity of 0.5, followed by 798 machines with a CPU capacity of 1, 126 machines with 0.25 and 32 machines with missing or unspecified CPU capacity ("NULL"). This distribution suggests that the cluster is standardised around medium-capacity nodes, probably because of a design choice to balance cost and performance. High-capacity machines (CPU = 1) are relatively limited, which may indicate they are reserved for more critical workloads. By contrast, lower-capacity machines serve as auxiliary nodes or for lightweight tasks. The concentration of machines around the 0.5 CPU class makes sure the scheduler efficiently manages resources while ensuring the cluster can handle a variety of workloads

without over-provisioning.

Q2. What is the percentage of computational power lost due to maintenance (a machine went offline and reconnected later)?

```
Q2: Lost computational power due to maintenance
Percentage of computational power lost: 1.89%
```

Figure 3: Result Q2

The analysis shows that 1.89% of the total computational power is lost due to machine maintenance, during which machines go offline and later reconnect to the cluster. This low percentage indicates that maintenance events have a minimal impact on overall cluster capacity, suggesting that maintenance activities are well-planned and managed to minimise disruption. In large-scale distributed systems, occasional downtime is unavoidable, but the observed value demonstrates that the cluster maintains high availability, with over 98% of computational power remaining accessible during operation.

Q3. Is there a class of machines, according to their CPU, that stands out with a higher maintenance rate, as compared to other classes?

```
Q3: Maintenance rate by CPU class
+----+------------------------+
|CPUs|avg_maintenance_duration|
+----+------------------------+
|0.25|     3.21459245980283E10|
| 0.5|    1.772325189637559E10|
| 1.0|    1.044356188280288...|
+----+------------------------+
```

Figure 4: Result Q3

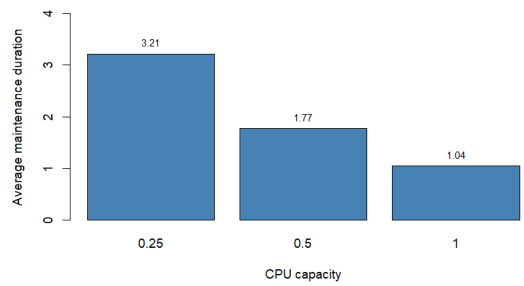

Figure 5: Average maintenance duration per CPU class

The analysis of maintenance duration per CPU class shows that machines with lower CPU capacity tend to experience longer maintenance events. This reveals that machines with lower CPU capacity are more vulnerable to longer maintenance periods, while higher-capacity machines experience shorter downtime. One possible explanation is that lower-end machines are older or less reliable, so they require more frequent or longer maintenance. In contrast, higher-end machines might be newer or more robust, leading to fewer disruptions.

Q4. What is the distribution of the number of jobs/tasks per scheduling class? Comment on the results.

```
Q4: Tasks per scheduling class
+----------------+------+
|scheduling_class| count|
+----------------+------+
|               0|415828|
|               1|183540|
|               2|102923|
|               3|  1935|
+----------------+------+
```

Figure 6: Result Q4

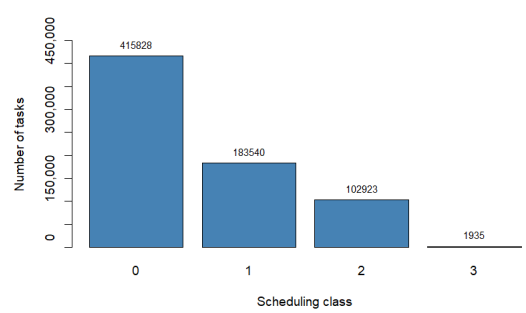

Figure 7: Number of tasks per Scheduling class

This distribution indicates that the cluster is primarily used for low-latency-sensitivity workloads, such as development. These tasks typically tolerate higher execution latency and benefit from flexible scheduling. As the scheduling class increases, the number of tasks decreases rapidly, probably because highly latency-sensitive tasks

are rare and controlled within the system.

Q5. Would you qualify the percentage of jobs/tasks that got killed or evicted as important?

```
 Q5: Percentage of tasks killed or evicted
Percentage: 8.28%
```

Figure 8: Result Q5

The analysis shows that 8.28% of jobs/tasks were killed or evicted. This is a moderate proportion, meaning that roughly 1 in 12 tasks does not complete successfully due to priority or eviction. While it is not a majority, this percentage is still important because it can impact the cluster efficiency and job completion times, particularly for high-priority or latency-sensitive tasks.

From a cluster management perspective, this indicates that evictions are an important factor to monitor, as they can reflect over-commitment or scheduling inefficiencies. Therefore, even though most tasks are completed successfully, the 8.28% that are killed or evicted should not be ignored in environments with critical or production workloads.

Q6. Do tasks with a low scheduling class have a higher probability of being evicted?

```
 Q6: Eviction probability by scheduling class
+----------------+--------------------+
|scheduling_class|       prob_eviction|
+----------------+--------------------+
|               0|0.001060534644131...|
|               1|0.001618175874468...|
|               2|0.006150228811830...|
|               3| 0.01808785529715762|
+----------------+--------------------+
```

Figure 9: Result Q6

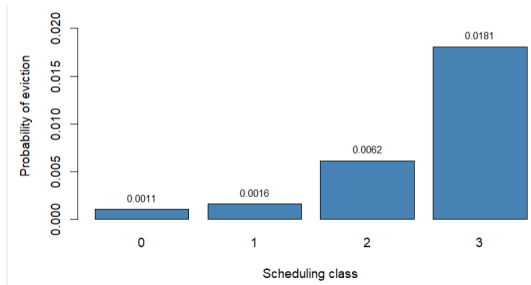

Figure 10: Probability of task eviction by Scheduling class

The results show that tasks with a lower scheduling class do not consistently have the highest probability of eviction. Contrary to what we expected, scheduling class 0 tasks does experience evictions. Their eviction probability (0.001060) is lower than that of scheduling classes 2 and 3. This suggests that evictions are not caused only by low scheduling class, but by other factors as well.

Q7. In general, do tasks from the same job run on the same machine? Comment on the observed locality strategy and its pros and cons.

```
 Q7: Task locality (Distinct machines per job)
+---------------+-----+
|unique_machines|count|
+---------------+-----+
|              1| 4619|
|              2|  273|
|              3|   40|
|              4|   21|
|              5|   47|
+---------------+-----+
only showing top 5 rows
```

Figure 11: Result Q7

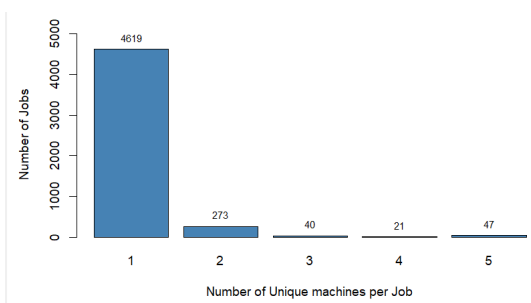

Figure 12: Number of Jobs by Number of Machines used

The analysis of task distribution per job shows that most jobs run their tasks on a single machine: 4,619 jobs have all their tasks executed on one machine, 273 jobs use two machines and the remaining jobs use more machines.

This shows that the cluster scheduler has a strong locality strategy, where tasks from the same job are placed on the same machine.

Pros of this approach:

- Tasks can access cached data or intermediate results without network transfer, reducing execution time.
- Concentrating tasks on one machine reduces cross-machine communication.
- It is easier to monitor and control resources for jobs concentrated on fewer machines.

Cons of this approach:

- Jobs concentrated on a single machine may conflict with each other, potentially leading to resource saturation.
- If a machine fails, all tasks from that job may be affected, requiring rescheduling and increasing job completion time.

Q8. Are the tasks that request the more resources the one that consume the more resources?

```
 Q8: Correlation between CPU Request and CPU Usage
Correlation between CPU Requested and Used: 0.3578367447272269
```

Figure 13: Result Q8

The analysis shows that the correlation between requested and consumed resources is 0.358. This indicates a moderate positive relationship, but it is also not a perfect match. The result suggests that resource requests are an imperfect predictor of actual usage. For the cluster scheduler, this implies that optimising task placement and resource allocation requires careful monitoring of real consumption more than relying only on requested amounts.

Q9. Can we observe correlations between peaks of high resource consumption on some machines and task eviction events?

```
 Q9: Correlation between machine max usage and eviction count
Correlation: -0.024172573157247267
```

Figure 14: Result Q9

The analysis shows that the correlation between peaks of high resource consumption and task eviction events is -0.024, which is very close to zero. This indicates that there is almost no linear relationship between high resource usage peaks on a machine and the occurrence of evictions. In other words, task evictions do not appear to be directly triggered by short-term spikes in resource consumption.

Q10. How often does it happen that the resources of a machine are over-committed?

```
 Q10: How often are machines over-committed?
Overloaded instances: 3 out of 10217149
```

Figure 15: Result Q10

The analysis shows that over-commitment of machine resources is extremely rare in this cluster. Out of 10,217,149 observations, only 3 instances were recorded where a machine's resource usage exceeded its available capacity. This corresponds to a percentage of approximately 0.00003%, which is negligible. Such a low occurrence indicates that the scheduler effectively prevents resource over-commitment, ensuring that tasks are allocated to machines in a way that avoids exceeding their CPU or memory limits.

3.2 Additional questions

Q11. What is the distribution of job sizes (number of tasks per job) in the cluster?

The analysis of job sizes, measured by the number of tasks per job, shows a highly uneven distribution. The majority of jobs, 4,796, consist of a single task, indicating that most jobs are small and likely simple or short-lived workloads. Jobs with 2 to 4 tasks are less frequent and the counts are dropping, while some intermediate sizes

(e.g., 10 tasks) and larger sizes (e.g., 14–15 tasks) appear rarely. While bigger multi-task activities represent more complex calculations that require task parallelism throughout the cluster, the majority of single-task jobs probably indicate a combination of development, testing or small-scale production workloads. This distribution demonstrates the cluster's capacity to effectively handle a combination of mostly small jobs with occasional larger workloads.

```
 Q11: Job Size Distribution
+---------+-----+
|num_tasks|count|
+---------+-----+
|        1| 4796|
|        2|  279|
|        3|   36|
|        4|   14|
|        5|   41|
|        6|   11|
|        7|   17|
|        8|    7|
|        9|   49|
|       10|    5|
|       11|  250|
|       12|   18|
|       13|    8|
|       14|   36|
|       15|  128|
+---------+-----+
only showing top 15 rows
```

Figure 16: Result Q11

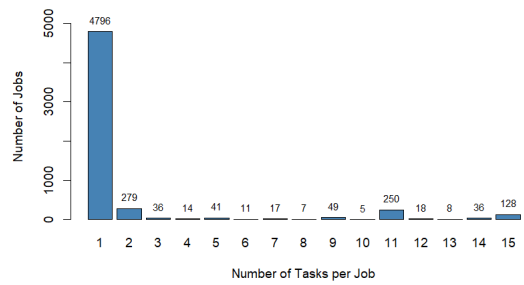

Figure 17: Distribution of Job sizes

Q12. Are tasks that run on multiple machines more likely to be killed than those running on a single machine?

The analysis shows an important difference in the probability of task termination depending on task distribution. Tasks that run on a single machine have a kill probability of 0.191, whereas tasks distributed across multiple machines have a much higher kill probability of 0.779. Out of 6,207 total jobs analysed, 4,619 were single-machine jobs and 1,588 were distributed across multiple machines. These results show that tasks using multiple machines are far more likely to be killed. This is likely due to increased coordination complexity, higher risk of resource conflict and greater exposure to failures across multiple nodes. In contrast, tasks limited to a single machine benefit from locality, reduced scheduling overhead and lower exposure to cross-machine failures, making them more stable.

```
 Q12: Job locality vs. Kill probability
+--------------+-------------------+----------+
|is_distributed|probability_of_kill|total_jobs|
+--------------+-------------------+----------+
|         false|0.19116691924659018|      4619|
|          true| 0.7789672544080605|      1588|
+--------------+-------------------+----------+
```

Figure 18: Result Q12

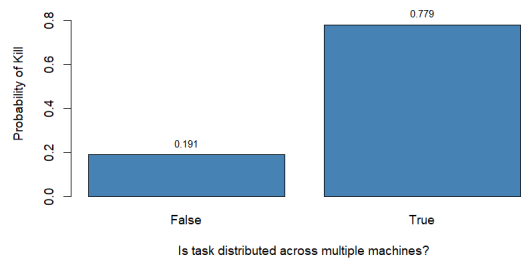

Figure 19: Probability of Task kill by Distribution

3.3 Extra work

3.3.1 Working in Google Cloud

As additional work, we deployed our analysis pipeline on Google Cloud Platform using a Dataproc cluster with two worker nodes. We first validated our approach by running the same PySpark code on the Google Cluster 2011 dataset, modifying only the dataset paths. The code executed successfully, confirming the correctness and portability of our implementation.

We then extended the study to the Google Cluster 2019 dataset. Due to schema and structural differences between the two traces, we adapted the code by renaming columns, handling nested fields and mapping collections and instances to jobs and tasks, respectively. The analysis was carried out on a subset of the dataset, in particular shards 105 to 109 and all jobs were only run on this selected fraction of the data in order to guarantee reliability and lower computational costs. After these adjustments, we ran the same set of analytical questions on the 2019 data.

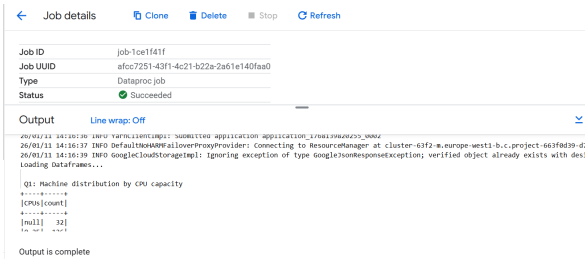

Figure 20: A job run successfully

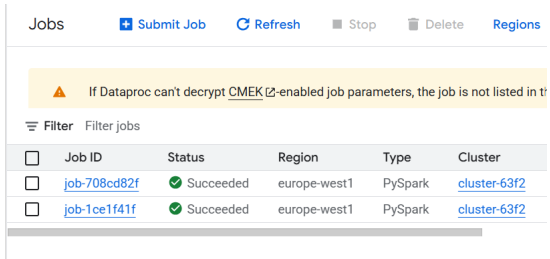

Figure 21: List of jobs in a google cluster

Analysis of Results on the 2019 Dataset

Q1. What is the distribution of the machines according to their CPU capacity?

```
Q1: Machine distribution by CPU capacity
+-----------+-----+
|       CPUs|count|
+-----------+-----+
|       null|  860|
| 0.38671875|  843|
|0.591796875| 1160|
|0.708984375|  888|
|0.958984375|  635|
|        1.0| 8066|
+-----------+-----+
```

Figure 22: Result from 2019 dataset to Q1

The CPU capacity distribution in the 2019 cluster is dominated by high-capacity machines, with most nodes having a CPU value of 1.0. Several intermediate capacity classes exist, indicating some variation, but the infrastructure is primarily optimised for performance-intensive workloads rather than low-capacity nodes.

Q2. What is the percentage of computational power lost due to maintenance?

```
Q2: Lost computational power due to maintenance
Percentage of computational power lost: 97.00%
```

Figure 23: Result from 2019 dataset to Q2

The analysis indicates a very high percentage of lost computational power (97%) due to maintenance. This value is significantly larger than expected and likely reflects differences in how maintenance events and machine availability are recorded in the 2019 dataset, rather than actual downtime.

Q3. Is there a class of machines, according to their CPU, that stands out with a higher maintenance rate, as compared to other classes?

Average maintenance duration increases with CPU capacity. Machines with higher CPU capacities tend to experience longer maintenance durations, which may be explained by the complexity and criticality of maintaining more powerful machines.

```
Q3: Maintenance rate by CPU class
+-----------+------------------------+
|       CPUs|avg_maintenance_duration|
+-----------+------------------------+
|       null|      2887754.4197674417|
| 0.38671875|    5.329381174550089E11|
|0.591796875|    5.946836385057219E11|
|0.708984375|    6.068480759094696E11|
|0.958984375|    6.268158187788864E11|
|        1.0|    6.062445426070458E11|
+-----------+------------------------+
```

Figure 24: Result from 2019 dataset to Q3

Q4. What is the distribution of the number of jobs/tasks per scheduling class?

```
Q4: Tasks per scheduling class
+----------------+-------+
|scheduling_class|  count|
+----------------+-------+
|               0|2314878|
|               1|2044158|
|               2| 478648|
|               3| 162752|
+----------------+-------+
```

Figure 25: Result from 2019 dataset to Q4

Most tasks belong to lower scheduling classes, while higher scheduling classes are less frequent. This indicates that the majority of workloads are non-latency-critical, with only a small fraction requiring latency guarantees.

Q5. Would you qualify the percentage of jobs/tasks that got killed or evicted as important?

```
Q5: Percentage of tasks killed or evicted
Percentage: 22.98%
```

Figure 26: Result from 2019 dataset to Q5

Approximately 22.98% of tasks were killed or evicted. This relatively high value suggests more aggressive resource management and pre-emption policies in the 2019 cluster compared to earlier traces.

Q6. Do tasks with a low scheduling class have a higher probability of being evicted?

```
Q6: Eviction probability by scheduling class
+----------------+------------------+
|scheduling_class|     prob_eviction|
+----------------+------------------+
|               0|0.2323409700208823|
|               1|0.2260363435703111|
|               2|0.1946440808276646|
|               3|0.2401199370821864|
+----------------+------------------+
```

Figure 27: Result from 2019 dataset to Q6

Eviction probabilities are comparatively high across all scheduling classes, with no apparent progressive decrease for higher classes. This suggests that scheduling class alone does not fully protect tasks from eviction in the 2019 system.

Q7. In general, do tasks from the same job run on the same machine?

```
Q7: Task locality (Distinct machines per job)
+---------------+-----+
|unique_machines|count|
+---------------+-----+
|              0|13025|
+---------------+-----+
```

Figure 28: Result from 2019 dataset to Q7

All observed jobs ran on a single machine, indicating a strong locality strategy. This approach reduces communication overhead but may limit scalability and fault tolerance.

Q8. Are the tasks that require more resources the one that consume the most resources?

```
Q8: Correlation between CPU Request and CPU Usage
Correlation between CPU Requested and Used: 0.6191643774432792
```

Figure 29: Result from 2019 dataset to Q8

A moderate-to-strong positive correlation (≈ 0.62) is observed, indicating that tasks requesting more CPU generally consume more CPU, though overestimation and underutilization still exist.

Q9. Can we observe correlations between peaks of high resource consumption on some machines and the task eviction events?

```
Q9: Correlation between machine max usage and eviction count
Correlation: 0.05115691698685493
```

Figure 30: Result from 2019 dataset to Q9

The correlation is very weak (≈ 0.05), suggesting that eviction events are not driven by peak CPU usage alone, but rather by other scheduling or policy-based factors.

Q10. How often does it happen that the resources of a machine are over-committed?

```
Q10: How often are machines over-committed?
Overloaded instances: 0 out of 3190908
```

Figure 31: Result from 2019 dataset to Q10

No over-commitment events were observed. This indicates strict enforcement of resource limits and conservative scheduling in the 2019 cluster.

Q11. What is the distribution of job sizes in the cluster?
The distribution of job sizes in the 2011 and 2019 datasets is quite similar. The majority of jobs in both traces are single tasks and as the number of tasks rises, the number of jobs decreases, suggesting a steady workload structure across time.

Q12. Are tasks that run on multiple machines more likely to be killed than those running on a single machine?

Only non-distributed jobs were observed, with a low probability of being killed ($\approx 2\%$), reinforcing the idea that strong locality is associated with higher job stability.

```
Q11: Job Size Distribution
+---------+-----+
|num_tasks|count|
+---------+-----+
|        1|98570|
|        2|12392|
|        3| 7938|
|        4| 5957|
|        5| 4900|
|        6| 4055|
|        7| 3385|
|        8| 2749|
|        9| 2189|
|       10| 1767|
|       11| 1367|
|       12| 1055|
|       13|  920|
|       14|  719|
|       15|  612|
+---------+-----+
only showing top 15 rows
```

Figure 32: Result from 2019 dataset to Q11

```
Q12: Job locality vs. Kill probability
+--------------+--------------------+----------+
|is_distributed| probability_of_kill|total_jobs|
+--------------+--------------------+----------+
|         false|0.020038387715930903|     13025|
+--------------+--------------------+----------+
```

Figure 33: Result from 2019 dataset to Q12

Google Cluster 2011 vs. 2019

When comparing the two datasets, it is clear how cluster design and workload characteristics have evolved. More machine capacity diversity, stricter resource enforcement and a higher focus on task localisation are all found in the 2019 cluster. The 2019 data shows stricter scheduling and eviction rules, potentially due to higher usage and more dynamic workloads, while the 2011 sample showed relatively low task eviction rates and minor maintenance impact.

Additionally, a stronger correlation between requested and consumed CPU indicates that task behaviour in 2019 is more predictable in terms of resource use. Overall, the comparison shows how large-scale cluster management has evolved over time, with a shift towards rigorous resource control, increased predictability and simplified execution patterns.

3.3.2 Studying Other Datasets

As additional work, we extended our analysis to the Azure Public Dataset (2019) to contrast Google's infrastructure (IaaS) with Azure's serverless environment (FaaS). We adapted our PySpark application to process the Azure traces and successfully executed the analysis on a local Spark-configured Virtual Machine. This confirmed the pipeline's portability and allowed us to compare Google's resource-intensive batch processing with Azure's event-driven workloads.

Q1. How many functions are in this dataset? The Azure 2019 dataset study shows 46,412 distinct serverless

```
Q1: Total number of functions
Count: 46412
```

Figure 34: Result from Azure dataset to Q1

functions. This represents the "population size" of the study. Unlike the Google dataset which counts physical machines, this counts individual pieces of deployed code (FaaS), showing that the serverless environment is fragmented with tens of thousands of small components.

Q2. What triggers these functions? This result groups functions by how they are activated. The most common

```
 Q2: Types of triggers
+--------------+-----+
|       Trigger|count|
+--------------+-----+
|         timer|17405|
|          http|15933|
|         queue| 6924|
|orchestration| 3287|
|         event| 1245|
|       storage|  949|
|        others|  669|
+--------------+-----+
```

Figure 35: Result from Azure dataset to Q2

triggers are Timers (37.5%) and HTTP requests (34.3%). This suggests that the two primary use cases for Azure Functions are scheduled background tasks (like cleanups or backups) and responding to web traffic (APIs). "Event" triggers are rare by count (only $\approx$ 2.7%).

Q3. Calculate total invocations per function?
This table reveals a big bias in resource consumption. The single busiest function (a Queue trigger) was invoked over 127 million times (Figure 36). The top 10 list is led by Queue and Event triggers, proving that while "Timers" are the most common type of function (from Q2), they are not the heaviest users. A small number of high-throughput functions drive the vast majority of the cloud's traffic.

Q4. Which trigger type creates the most load?
This compares the average workload of different trigger types. There is a big difference in intensity: Event Triggers - Extremely high volume, averaging $\approx$ 240,000 invocations per function. Timer Triggers - Very low volume, averaging only $\approx$ 948 invocations per function (Figure 37). "Event" functions are rare but work very hard, while "Timer" functions are common but work very little.

```
 Q3: Calculating total invocations
Top 10 busiest functions:
+--------------------+-------+----------------+
|        HashFunction|Trigger|total_invocations|
+--------------------+-------+----------------+
|8203ff88388384a6f...|  queue|       127471685|
|5315be05fc3b21a3f...|   http|        83591132|
|dd833bb70b3a57caa...|  queue|        48947822|
|93d4c31373200d742...|  event|        24011909|
|ea885ffca6338a2b7...|  event|        22457375|
|063ba6f3c1d425f6f...|  event|        22030820|
|2b373145c4fa2c344...|  event|        20918946|
|fd87c379a4f2dac5b...|  event|        16743959|
|04605a37a22e05096...|  event|        16346524|
|308ecb6c8592f6f83...|  queue|        15937049|
+--------------------+-------+----------------+
only showing top 10 rows
```

Figure 36: Result from Azure dataset to Q3

```
 Q4: Average load by trigger type
+-------------+------------------+---------+
|      Trigger|         avg_calls|max_calls|
+-------------+------------------+---------+
|        event|240352.70522088354| 24011909|
|        queue| 54671.90944540728|127471685|
|         http|12301.149814849683| 83591132|
|      storage| 8070.704952581665|  1963422|
|orchestration| 3362.828110739276|  1448320|
|       others|1168.8161434977578|   142815|
|        timer| 948.4106291295604|    86700|
+-------------+------------------+---------+
```

Figure 37: Result from Azure dataset to Q4

Q5. Which Application is the biggest?

```
 Q7: Top applications (Apps with most invocations)
+--------------------+--------------+---------------+
|             HashApp|function_count|app_total_calls|
+--------------------+--------------+---------------+
|94409f2485ebd997a...|             1|      127471685|
|734bb9a04a1d2e691...|             8|       91582836|
|4b75f2532af503341...|             5|       49036404|
|228ef3c9f06cde19b...|             2|       46042729|
|dd3b9ba973ba58819...|             1|       22457375|
+--------------------+--------------+---------------+
only showing top 5 rows
```

Figure 38: Result from Azure dataset to Q5

The #1 most active application consists of only 1 single function, yet it processed 127 million calls. This indicates a "monolithic" or simple pipeline design. The #2 application contains 8 functions, suggesting a "microservices" architecture where work is split across multiple components. This demonstrates how the complexity of serverless applications varies widely in complexity, from single-purpose scripts to complex multi-function systems.

Comparison: Google Cluster vs. Azure Functions

In this section, we compare the infrastructure-level (IaaS) insights from the Google Cluster dataset with the application-level (FaaS) patterns observed in the Azure Functions dataset.

The Scale Comparison

While Google focuses on physical hardware and tasks, Azure focuses on abstract function executions.

Feature	**Google Cluster (IaaS)**	**Azure Functions (FaaS)**
Unit of Analysis	Machines & Tasks (Hardware level)	Functions (Application level)
Total Count	~12,600 Machines (from Figure 1)	46,412 Functions (from Figure 34)
Dominant Resource	Small/Medium VMs (0.5 CPU is 92% of fleet)	High-Frequency Triggers (Timers/HTTP)
Takeaway	You are seeing the "Data Center" view.	You are seeing the "Developer" view.

Table 1: Comparison of Scale between Google and Azure datasets.

Analysis:

- **Google:** The infrastructure is uniform. Our results (Figure 1) show that 11,659 machines (the vast majority) have exactly 0.5 Normalized CPU capacity. This standardization helps Google pack jobs tightly.
- **Azure:** The application layer is inconsistent. We found 4× more "Functions" than Google has "Machines" demonstrating that serverless architectures create a massive dispersion of little components.

Workload Distribution

Google (Figure 11)

- 90%+ of jobs run on exactly 1 machine.
- Only a little fraction (40 jobs) run on 3 machines.

Most Google jobs are actually small, independent tasks (likely MapReduce shards or simple scripts), rather than massive MPI super-computing jobs.

Azure (Figure 36) and (Figure 11)

- The Top 1% of functions consume almost all the resources.
- The #1 application has only 1 function but was triggered 127 million times.

A few large apps power serverless billing, while thousands of others—mostly "Timer" functions—are inactive.

Summary

Comparing the Google and Azure datasets reveals the fundamental difference between IaaS and FaaS. The Google trace (Figure 8) shows that infrastructure reliability is probabilistic, with an 8.28% task kill rate needing robust fault-tolerance in user code. In contrast, the Azure trace (Figure 37) demonstrates that application load is unbalanced, with "Event" triggers generating 250× more volume than "Timer" triggers. While Google optimizes for packing efficiency (standardized 0.5 CPU machines), Azure optimizes for elasticity (handling 127M invocations on a single function).

4 Challenges and Technical Issues

During the analysis, we encountered several challenges and technical issues that required handling. First, understanding the structure and content of the datasets proved to be more complex than we expected. Both the 2011 and 2019 Google Cluster datasets contain multiple tables with different schemas, nested fields and varying formats for timestamps and resource usage, which required us to map fields to maintain consistency across analyses.

Second, while designing additional original questions, we initially planned to compare the behaviour of jobs submitted during standard working hours (9:00–17:00) versus outside these hours. However, we realised that the timestamps in the dataset are not in a standard datetime format, making it difficult to determine the hour of submission. This limitation forced us to adjust our analysis and propose alternative questions.

Third, accessing additional datasets such as Alibaba and Microsoft Azure presented difficulties. These datasets are no longer directly accessible via *gsutil*, so we had to download them locally, which increased the time and complexity of preprocessing and data loading.

Finally, setting up and configuring cloud clusters using Google Dataproc presented technical obstacles with worker node configuration and resource allocation. Some clusters failed to start or required reconfiguration, which caused delays in running the analysis and required repeated troubleshooting.

Despite these challenges, we were able to successfully preprocess the datasets, adapt our analysis to the available data and perform meaningful evaluations across multiple cloud traces.

5 Conclusion

In this work, we analyzed large-scale cluster traces to study resource usage, scheduling behavior and system reliability in cloud computing environments. The analyses were initially conducted on a representative subset of the data and later scaled using cloud infrastructure.

The Google Cluster Data 2011 analysis revealed a comparatively homogeneous cluster with moderate task eviction rates and little maintenance impact. Only a minor link was found between resource requests and actual utilization, suggesting users' careful overestimation and low predictability. The scheduler demonstrated a balanced strategy between locality and distribution, supporting both small and multi-task jobs.

The Google Cluster Data 2019 showed a clear evolution in cluster management. The infrastructure became more heterogeneous, task locality was enforced and eviction rates increased significantly, suggesting more aggressive resource management and pre-emption policies. At the same time, the stronger relation between requested and consumed CPU reflects improved workload characterization and scheduling maturity.

Deploying the analysis on a Google Cloud Dataproc cluster confirmed the scalability of our Spark-based pipeline. Running the same code on both datasets, with only schema-level adaptations, demonstrated its flexibility and reusability. The use of cloud resources also allowed efficient processing of larger data volumes.

Extending the analysis to the Azure serverless dataset provided a different viewpoint to traditional IaaS clusters. The results showed the highly fragmented nature of serverless platforms, where a small number of event-driven functions generate the majority of system load. Overall, this comparative study illustrates how cloud systems have evolved toward stricter resource control, specialized execution models and more dynamic workload management.

References and Related Research

Reiss, C., Tumanov, A., Ganger, G. R., Katz, R. H., & Kozuch, M. (n.d.). Heterogeneity and dynamicity of clouds at scale. *Proceedings of the Third ACM Symposium on Cloud Computing—SoCC*, *12*.

Alam, M., Shakil, K. A., & Sethi, S. (2016). Analysis and clustering of workload in google cluster trace based on resource usage. *2016 IEEE Intl Conference on Computational Science and Engineering (CSE) and IEEE Intl Conference on Embedded and Ubiquitous Computing (EUC) and 15th Intl Symposium on Distributed Computing and Applications for Business Engineering (DCABES)*, 740–747. https://doi.org/10.1109/CSE-EUC-DCABES.2016.271

Tirmazi, M., Barker, A., Deng, N., Haque, M. E., Qin, Z. G., Hand, S., Harchol-Balter, M., & Wilkes, J. (2020). Borg: The next generation. *Proceedings of the fifteenth European conference on computer systems*, 1–14.

Bappy, F. H., Islam, T., Zaman, T. S., Hasan, R., & Caicedo, C. (2023). A deep dive into the google cluster workload traces: Analyzing the application failure characteristics and user behaviors. *2023 10th International Conference on Future Internet of Things and Cloud (FiCloud)*, 103–108. https://doi.org/10.1109/FiCloud58648.2023.00023

Shahrad, M., Fonseca, R., Goiri, I., Chaudhry, G., Batum, P., Cooke, J., Laureano, E., Tresness, C., Russinovich, M., & Bianchini, R. (2020). Serverless in the wild: Characterizing and optimizing the serverless workload at a large cloud provider. *2020 USENIX annual technical conference (USENIX ATC 20)*, 205–218.

Armbrust, M., Xin, R. S., Lian, C., Huai, Y., Liu, D., Bradley, J. K., Meng, X., Kaftan, T., Franklin, M. J., Ghodsi, A., et al. (2015). Spark sql: Relational data processing in spark. *Proceedings of the 2015 ACM SIGMOD international conference on management of data*, 1383–1394.

Apache Software Foundation. (2025). *Pyspark tutorials* [Accessed: 2026-01-14]. The Apache Software Foundation. https://spark.apache.org/docs/latest/api/python/tutorial/index.html